

Assignment 7: Supersampling and Antialiasing

作者：王洵

1 实验原理

1.1 混叠与超采样重构

离散光栅图像将连续的二维图像平面划分成有限个像素正方形。理想情形下，像素 (i, j) 的颜色应对应该像素支撑域上连续辐射亮度场 $L(u, v)$ 与重构核 f 的卷积

$$C(i, j) = \iint_{\mathbb{R}^2} L(u, v) f(u - (i + 0.5), v - (j + 0.5)) du dv,$$

其中 (u, v) 为归一化屏幕坐标， $(i + 0.5, j + 0.5)$ 为像素中心。光线追踪 (ray tracing) 对每个像素仅沿单一视线求值时，等价于用 Dirac 脉冲代替积分核；当场景中存在几何边界、棋盘格等高频内容时，采样率低于 Nyquist 频率，频谱混叠 (aliasing) 表现为锯齿边缘与莫尔纹 (Moire pattern)。

超采样 (supersampling) 在像素内取 N 个位置 (u_k, v_k) ，分别追踪得到颜色 $\mathbf{c}_k$ ，再以离散加权和逼近上述积分：

$$C(i, j) \approx \frac{\sum_k w_k \mathbf{c}_k}{\sum_k w_k}, \quad w_k = f(u_k - (i + 0.5), v_k - (j + 0.5)).$$

分母归一化使滤波器增益恒为 1，平坦区域颜色不被核的整体尺度改变。像素内偏移 $\delta = (\delta_x, \delta_y) \in [0, 1]^2$ 将整数像素坐标映射到屏幕坐标

$$u = \frac{i + \delta_x}{W}, \quad v = \frac{j + \delta_y}{H},$$

再由相机投影生成对应射线，使多样本覆盖该像素对应的立体角。

1.2 采样图案

样本位置的空间分布决定了对高频误差的抑制方式。设每像素样本数为完全平方数 $N = d^2$ 。

均匀网格采样 (uniform sampling) 将像素划分为 $d \times d$ 个等大正方形单元，取各单元中心

$$\delta_{ix, iy} = \left(\frac{ix + 0.5}{d}, \frac{iy + 0.5}{d} \right), \quad ix, iy \in \{0, \dots, d - 1\}.$$

该模式覆盖均匀、方差低；规则点阵与周期性纹理相互作用时，易产生结构化混叠。

随机采样 (random sampling) 令 δ_x, δ_y 独立取自 $[0, 1]$ 上的均匀分布，破坏规则相位，将混叠能量转化为宽带噪声。视觉上噪声往往比规则伪影更易接受，但低样本数下方差较大。

抖动采样 (jittered sampling) 在均匀网格的每个单元内再引入独立均匀随机扰动 $\xi_x, \xi_y \in [0, 1]$ ：

$$\delta_{ix, iy} = \left(\frac{ix + \xi_x}{d}, \frac{iy + \xi_y}{d} \right).$$

各单元只包含一个样本，从而兼顾空间均匀性与相位随机性：既降低空洞与团簇带来的方差，又削弱规则网格与周期纹理之间的相干干涉。

1.3 重构滤波器

重构阶段以核函数 $f(x, y)$ 对邻域样本赋权， x, y 为相对输出像素中心的偏移（以像素边长为单位）。

盒式滤波 (box filter) 在轴对齐正方形支撑上取常数权重：

$$f_{\text{box}}(x, y) = \begin{cases} 1, & |x| \leq r \text{ 且 } |y| \leq r, \\ 0, & \text{otherwise,} \end{cases}$$

其中 r 为像素中心到边界的正交距离；当 $r = 0.5$ 时支撑为一个像素，等价于对该像素内样本等权平均。

帐篷滤波 (tent filter) 按到中心的欧氏距离线性衰减：

$$f_{\text{tent}}(x, y) = \max\left(0, 1 - \frac{\sqrt{x^2 + y^2}}{r}\right).$$

中心权重为 1，距离超过 r 处为零，靠近像素中心的样本贡献更大。

高斯滤波 (Gaussian filter) 采用径向基

$$f_{\text{gauss}}(x, y) = \begin{cases} \exp(-d^2/(2\sigma^2)), & d \leq 2\sigma, \\ 0, & d > 2\sigma, \end{cases} \quad d = \sqrt{x^2 + y^2},$$

其中 σ 为标准差；将支撑截断于 2σ ，在保留主瓣能量的同时限制计算邻域。

对支撑半径为 R 的滤波器，输出像素 (i, j) 汇集整数坐标落在 $[i - R, i + R] \times [j - R, j + R]$ 内的样本。图像边界处邻域被裁剪，归一化分母只对实际累加的权重求和，使边界像素仍得到合理亮度。样本数增加、核支撑适度扩大时，几何轮廓与高频纹理上的混叠逐步减弱。

2 程序设计与实现

2.1 总体架构

系统在既有光线追踪、网格加速与程序化材质之上，增加胶片缓冲、采样器与滤波器三个模块：

1. **胶片缓冲**：按宽度、高度与每像素样本数存储全部颜色样本；
2. **采样器**：给出像素内二维偏移；
3. **滤波器**：对邻域样本做加权重构，得到最终像素颜色。

命令行解析识别随机 / 均匀 / 抖动采样、盒式 / 帐篷 / 高斯滤波，以及采样图案与滤波权重的可视化选项。渲染流程先按采样器将射线追踪结果写入胶片，再滤波生成最终图像。

2.2 胶片缓冲与主渲染循环

每个样本保存像素内偏移 $\delta \in [0, 1]^2$ 与对应颜色：

```
class Sample {
    Vec2f position; // 像素内偏移 (0,0)→(1,1)
    Vec3f color;
};
```

胶片为每个像素分配固定数量的样本槽。主循环对每个像素与每个样本依次执行：由采样器取得偏移，映射为屏幕坐标后生成相机射线，追踪得到颜色并写入胶片：

```
for (int y = 0; y < height; y++) {
    for (int x = 0; x < width; x++) {
        for (int s = 0; s < numSamples; s++) {
            Vec2f offset = sampler->getSamplePosition(s);
            float u = (x + offset.x()) / (float)width;
            float v = (y + offset.y()) / (float)height;
            Ray ray = generateCameraRay(u, v);
            Vec3f color = rayTracer->traceRay(
                ray, camera->getTMin(), 0, 1.0f, 1.0f, hit);
            film.setSample(x, y, s, offset, color);
        }
    }
}
```

未指定采样选项时，默认均匀采样且每像素仅一个样本，偏移取像素中心 (0.5, 0.5)。固定伪随机种子后，随机采样与抖动采样的结果可复现。若启用采样可视化，则将放大后的采样点分布写成图像。

全部样本写完后，对每个输出像素调用滤波器得到颜色：

```
for (int y = 0; y < height; y++) {
    for (int x = 0; x < width; x++) {
        Vec3f color = (filter != NULL)
            ? filter->getColor(x, y, &film)
            : averagePixelSamples(&film, x, y);
    }
}
```

```

        image->SetPixel(x, y, color);
    }
}

```

未指定滤波器时，对当前像素内样本做等权平均。

2.3 采样器模块

采样器保存每像素样本数，并按样本下标返回像素内二维偏移：

```

class Sampler {
public:
    virtual Vec2f getSamplePosition(int n) = 0;
protected:
    int numSamples;
};

```

随机采样器： δ_x, δ_y 独立取自 $[0, 1)$ ：

```

float x = rand() / (RAND_MAX + 1.0f);
float y = rand() / (RAND_MAX + 1.0f);
return Vec2f(x, y);

```

均匀采样器：要求 $N = d^2$ ，按网格索引取单元中心：

```

int ix = n % gridSize;
int iy = n / gridSize;
float x = (ix + 0.5f) / (float)gridSize;
float y = (iy + 0.5f) / (float)gridSize;

```

抖动采样器：在单元 (ix, iy) 内再叠加独立抖动：

```

float jx = rand() / (RAND_MAX + 1.0f);
float jy = rand() / (RAND_MAX + 1.0f);
float x = (ix + jx) / (float)gridSize;
float y = (iy + jy) / (float)gridSize;

```

2.4 滤波器模块

滤波器在支撑半径邻域内遍历样本，将样本位置换算为相对输出中心的偏移后加权平均：

```

for (int x = i - support; x <= i + support; x++) {
    for (int y = j - support; y <= j + support; y++) {
        if (x < 0 || x >= width || y < 0 || y >= height)
            continue;
        for (int n = 0; n < numSamples; n++) {
            Sample s = film->getSample(x, y, n);
            float fx = (x + s.getPosition().x()) - (i + 0.5f);
            float fy = (y + s.getPosition().y()) - (j + 0.5f);
            float w = getWeight(fx, fy);
            colorSum += s.getColor() * w;
            weightSum += w;
        }
    }
}
return colorSum * (1.0f / weightSum);

```

三种核的权重实现如下。

盒式滤波：

```

if (fabs(x) <= radius && fabs(y) <= radius)
    return 1.0f;
return 0.0f;

```

支撑半径取 $\lceil r \rceil$ 。

帐篷滤波：

```
float d = sqrtf(x * x + y * y);
if (d >= radius) return 0.0f;
return 1.0f - d / radius;
```

高斯滤波：

```
float d2 = x * x + y * y;
if (d2 > (2.0f * sigma) * (2.0f * sigma))
    return 0.0f;
return expf(-d2 / (2.0f * sigma * sigma));
```

支撑半径取 $\lceil 2\sigma \rceil$ 。若启用滤波可视化，则将核权重分布写成图像。

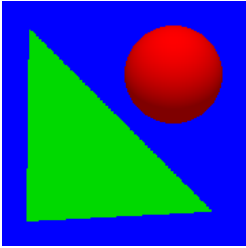
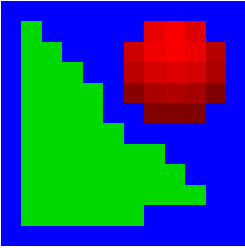
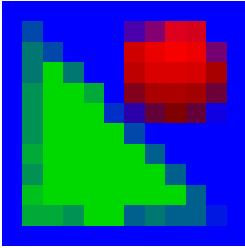
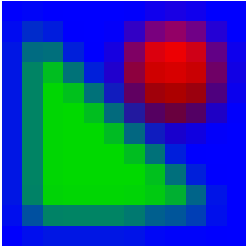
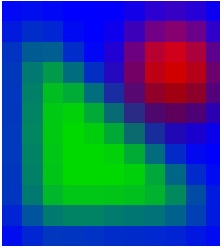
3 实验结果

测试命令行如：

```
# 球体与三角形; -size 图像分辨率, -output 输出图像
raytracer -input <球体与三角形场景> -size 180 180 -output output7_01.tga
# -render_samples 采样图案可视化与放大倍数, -random_samples 随机采样样本数
raytracer -input <球体与三角形场景> -size 9 9 -render_samples samples7_01a.tga 20 -random_samples 4
# -jittered_samples 抖动采样, -box_filter 盒式滤波半径
raytracer -input <棋盘格地板场景> -size 180 180 -output output7_02g.tga -jittered_samples 16 -box_filter 0.5
# -grid 网格分辨率, -shadows 阴影, -gaussian_filter 高斯滤波标准差
raytracer -input <大理石花瓶场景> -size 200 200 -output output7_03d.tga -grid 15 30 15 -shadows -jittered_samples 36 -gaussian_filter 0.5
```

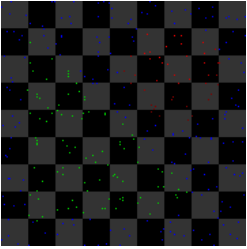
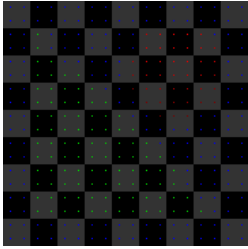
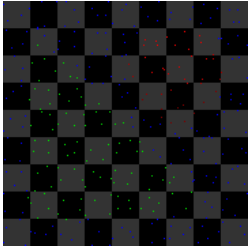
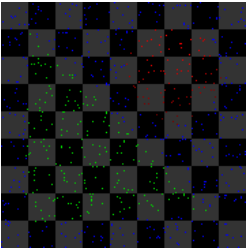
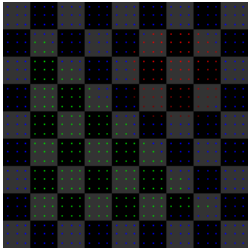
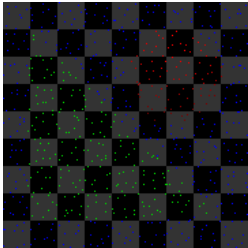
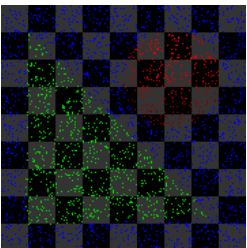
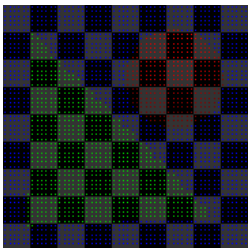
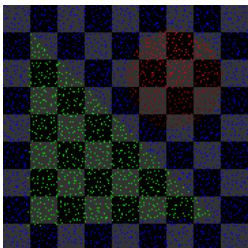
单样本渲染保留明显锯齿；增大每像素样本数并配合盒式 / 帐篷 / 高斯滤波后，几何边缘与棋盘格等高频纹理上的混叠明显减弱。

3.1 球体与三角形

单样本基准	低分辨率单样本	随机 + 盒式	均匀 + 帐篷	抖动 + 高斯
				
正常分辨率基准	锯齿明显	轮廓开始平滑	边缘更连续	视觉最平滑

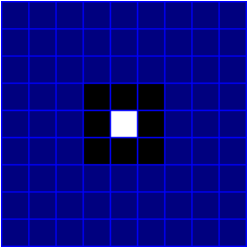
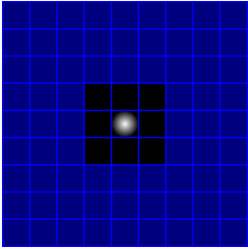
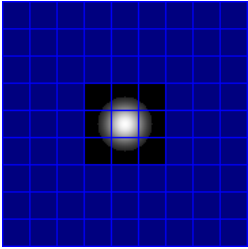
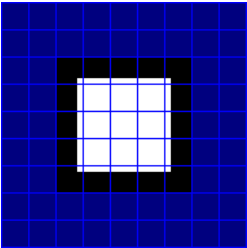
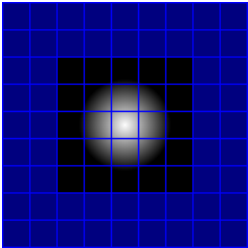
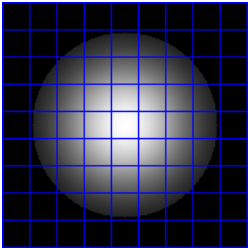
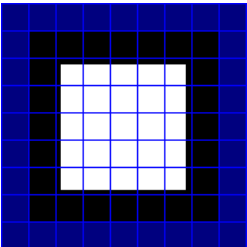
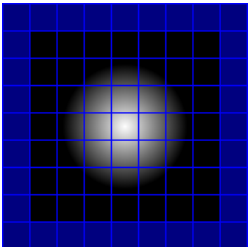
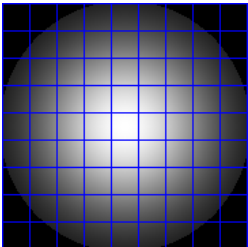
单样本低分辨率图锯齿明显；随机采样配合盒式滤波、均匀采样配合帐篷滤波、抖动采样配合高斯滤波后，球体与三角形轮廓逐步平滑。

采样图案（ 9×9 ，放大倍数 20）

	样本数 4	样本数 9	样本数 36
随机			
	散布无规则	密度上升	点彩接近色块
均匀			
	规则网格	规则网格	规则网格
抖动			
	单元内轻微偏移	单元内轻微偏移	单元内轻微偏移

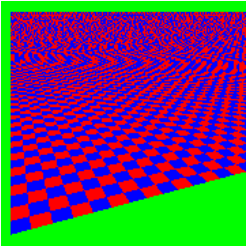
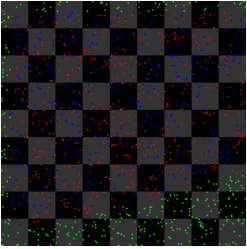
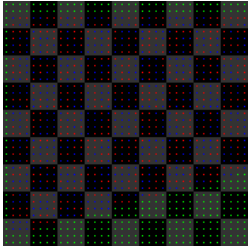
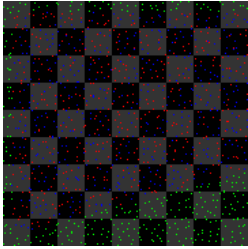
随机采样点散布无规则；均匀采样呈规则网格；抖动采样在各网格单元内轻微偏移。样本数增大时点密度上升，点彩效果更接近场景整体色块。

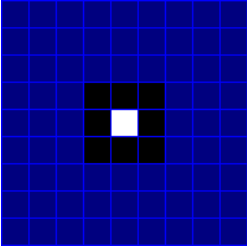
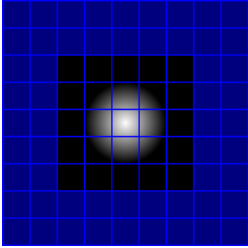
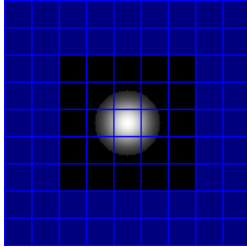
滤波权重可视化

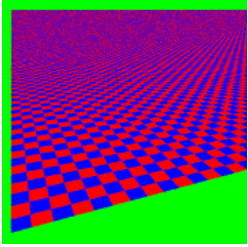
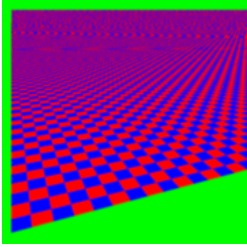
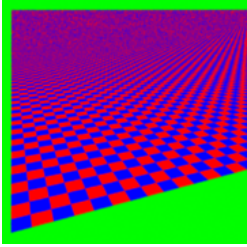
	参数 0.5	参数 1.7	参数 2.3
盒式			
帐篷	方形支撑	支撑扩大	支撑更大
帐篷			
高斯	径向线性衰减	圆盘扩大	圆盘更大
高斯			
	中心亮、外围渐暗	σ 增大	支撑覆盖更多邻域

盒式核呈方形支撑；帐篷核呈径向线性衰减的圆盘；高斯核中心亮、外围渐暗。半径或 σ 增大时支撑覆盖更多邻域像素。

3.2 棋盘格地板

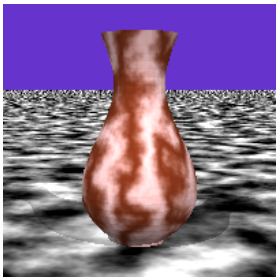
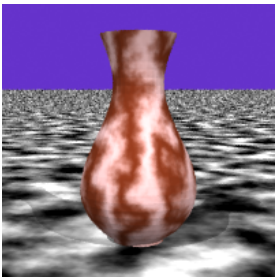
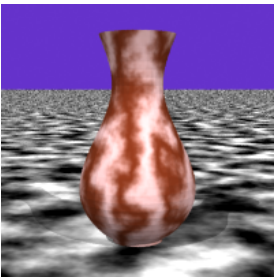
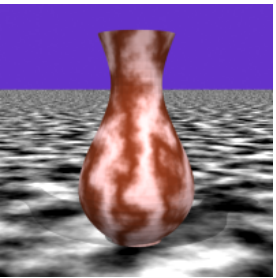
单样本基准	随机采样图案 (16 样本)	均匀采样图案	抖动采样图案
			
远处莫尔纹与边缘锯齿	随机点分布	规则网格	抖动网格

盒式滤波权重	帐篷滤波权重	高斯滤波权重
		

180 × 180、16 样本 随机 + 盒式	均匀 + 帐篷	抖动 + 高斯
		
远处格子更连续	规则伪影减弱	噪声与莫尔纹均较弱

单样本时远处棋盘格出现明显莫尔纹与边缘锯齿。16 样本超采样后，远处格子边界更连续；抖动采样配合帐篷或高斯滤波时，规则伪影与噪声都相对较弱。

3.3 大理石花瓶

单样本	4 抖动 + 高斯 $\sigma = 0.4$	9 抖动 + 高斯	36 抖动 + 高斯
			
轮廓与纹理锯齿清晰	边缘开始平滑	过渡更连续	轮廓与纹理最平滑

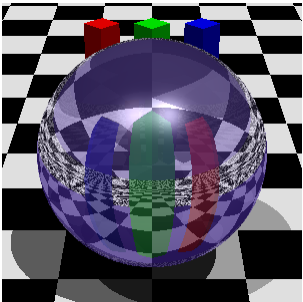
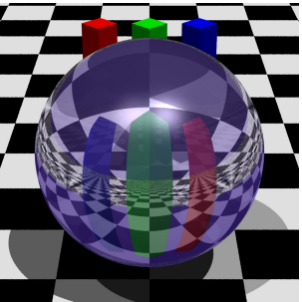
单样本时花瓶轮廓与大理石纹理边缘锯齿清晰；样本数增至 36 后，轮廓与纹理过渡更平滑，说明在复杂网格场景上超采样与高斯重构仍有效。

3.4 6.837 Logo

单样本	9 抖动样本 + 高斯滤波 ($\sigma = 0.4$)
	
字母边缘与高光锯齿明显	笔画轮廓更清晰

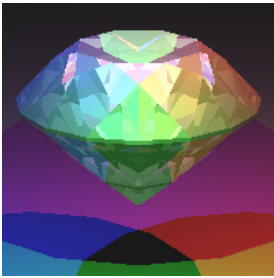
超采样后字母边缘与高光锯齿减轻，logo 笔画轮廓更清晰。

3.5 玻璃球

单样本	4 抖动 + 高斯 $\sigma = 0.4$	16 抖动 + 高斯 $\sigma = 0.4$
		
轮廓与焦散锯齿明显	边界开始连续	高光边界抖动减小

透明折射场景下，球面轮廓与焦散状细节在样本数增加后更连续，高光边界抖动减小。

3.6 多面体宝石

单样本	9 抖动样本 + 高斯滤波 ($\sigma = 0.4$)
	
棱边与内部亮斑锯齿明显	亮斑过渡更柔和

宝石棱边与内部亮斑在超采样后锯齿减弱，亮斑过渡更柔和。

4 问题记录

渲染场景出现了暗斑与发黑轮廓。原因是接入胶片缓冲后，之前的实现是先把整幅图填成背景再写命中像素，而超采样后每个样本都会参与处理，边缘附近的背景样本会参与滤波平均，从而导致问题发生。解决：在命中材质为空时直接写入场景背景色，问题解决。

5 总结

将每像素单次求值扩展为像素内多样本追踪，并以盒式、帐篷或高斯核邻域样本做归一化加权重构，从离散积分角度抑制几何边界与高频纹理引起的频谱混叠。随机、均匀与抖动三种采样图案分别偏向噪声化伪影、规则覆盖与二者折中；配合可配置的支撑半径与可视化，使锯齿与莫尔纹随样本数增加而减弱，各测试场景的边缘与纹理表现与带限重构的预期相符。

附录 (TODO list 记录)

- ☒ 集成胶片缓冲，将射线追踪颜色写入每像素多样本存储
- ☒ 实现采样器抽象接口与随机采样器，并支持命令行指定样本数
- ☒ 在主渲染循环中按采样偏移发射射线并写入胶片，默认单中心均匀采样
- ☒ 添加采样图案可视化
- ☒ 实现均匀采样器与抖动采样器，并固定随机种子使结果可复现
- ☒ 实现滤波器抽象接口，在支撑半径邻域内对样本做加权平均
- ☒ 实现盒式、帐篷与高斯滤波及对应命令行选项
- ☒ 添加滤波权重可视化，并在采样完成后输出滤波图像

Extra Credit

- ☐ Poisson 或 n-rook 采样图案
- ☐ 自适应采样
- ☐ Mitchell 滤波
- ☐ Perlin 噪声频率截断